

西北工业大学
Northwestern Polytechnical University

Object Oriented Programming (U10M12004) Group Project Report

Title	Chess Tournaments Management System
Group No.	10
Leader	HANA KOUIRITI YASMINE AIT HADDOU
Member(s)	RAPHAEL GIN CHENG TANG

May 08 2025

I. Introduction

The Chess Tournament Management System is a comprehensive Java based application developed to modernize, streamline, and enhance the management of chess tournaments. With the growing popularity of chess both online and offline, and the increasing demand for efficient, scalable tools to handle tournament logistics, the need for a robust digital solution has never been more apparent. Traditional methods of organizing chess tournaments often rely heavily on manual processes, pen-and-paper registrations, hand-crafted pairing tables, and fragmented communication channels, which are not only time-consuming but also prone to error and inefficiency. This system is designed to address these challenges head-on by offering an integrated platform that brings structure, reliability, and ease-of-use to tournament administration.

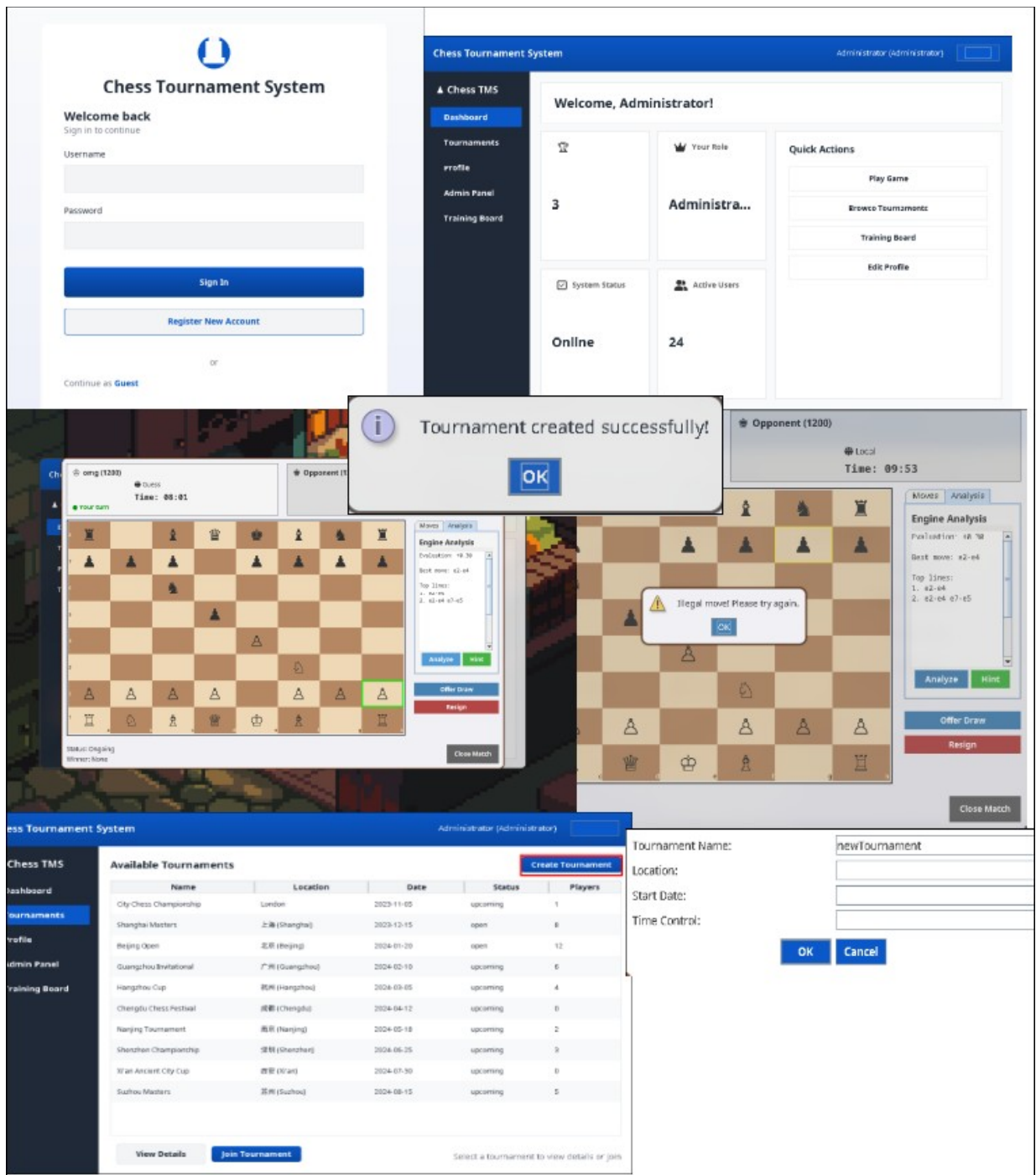
At its foundation, the system is engineered to automate the entire tournament lifecycle. This includes the creation of new tournaments, player enrollment and verification, match pairings based on tournament format (e.g., Swiss, round-robin), real-time match tracking, score entry, leaderboard updates, and the final tabulation of results. These features are encapsulated within a unified user interface that prioritizes clarity and usability, ensuring that users, whether they are administrators, referees, or players, can navigate the system with minimal training.

The architecture of the application emphasizes both performance and security. Leveraging Java's object-oriented capabilities and modular design patterns, the system is built to be maintainable and extensible, allowing for future enhancements such as online play integration, rating system support, or analytics dashboards. Moreover, the system adheres to industry-standard security protocols to safeguard user data and ensure the integrity of tournament information. This includes secure authentication mechanisms, access controls, and data validation processes to prevent unauthorized access or tampering.

A key strength of the Chess Tournament Management System lies in its adaptability. It is suitable for managing events of varying scales, from small in-person tournaments at local clubs to larger, multi-round regional or even national events. Its scalability is supported not only by its technical design but also by its intuitive interface, which lowers the barrier to entry for users of different technical backgrounds. This inclusive design approach allows tournament organizers and participants to focus on the competitive and strategic aspects of chess, rather than being burdened by logistical complexities.

As the chess community continues to expand, driven in part by increased online engagement, educational initiatives, and global outreach tools that enhance the organization and experience of tournaments become increasingly valuable. The Chess Tournament Management System positions itself as a forward-thinking solution in this space, offering a reliable framework for managing tournaments with precision, efficiency, and professionalism. In doing so, it not only supports the operational needs of chess events but also contributes to the broader mission of making competitive chess more accessible, transparent, and enjoyable for all involved.

Quick Preview of the GUI



II. Problem Statement

The organization and management of chess tournaments have traditionally presented numerous challenges that affect organizers, players, and spectators alike. These difficulties often lead to inefficiencies, errors, and suboptimal experiences for all involved.

Tournament Administration remains one of the most significant challenges. Organizers typically spend countless hours managing registrations, creating pairings, updating standings, and recording results. This manual approach is not only time-consuming but also highly prone to human error. A slight miscalculation in standings or an incorrect pairing can compromise the integrity of an entire tournament. Furthermore, last-minute changes, such as player withdrawals or rescheduling, create cascading complications that are difficult to manage on paper or with basic spreadsheets.

Match Oversight presents another critical challenge. In tournaments with multiple simultaneous games, ensuring fair play and proper rule application becomes increasingly difficult. Referees must monitor numerous matches, resolve disputes, and verify that time controls are being respected. Without a systematic approach, these responsibilities can quickly become overwhelming, potentially leading to overlooked infractions or inconsistent rule application.

From the **Player Experience** perspective, participants often struggle to access up-to-date information about their standings, upcoming matches, and tournament progress. Players need reliable access to their personal statistics, match histories, and current tournament details to prepare effectively and fully engage with the competition. The lack of immediate access to this information can lead to confusion, missed matches, and a diminished overall experience.

Data Management challenges extend beyond the tournament itself. Maintaining comprehensive records of players, matches, and results requires a structured system with appropriate validation mechanisms. Historical data is invaluable for rating calculations, player development tracking, and tournament planning, yet many organizers lack the tools to effectively capture and utilize this information.

Finally, **Access Control** presents a significant security and operational challenge. Different stakeholders require varying levels of access to tournament functions and data. Administrators need complete control, referees require oversight capabilities for their assigned matches, and players should only have access to their own information and public tournament data. Without proper role-based access controls, tournaments risk unauthorized changes or privacy breaches.

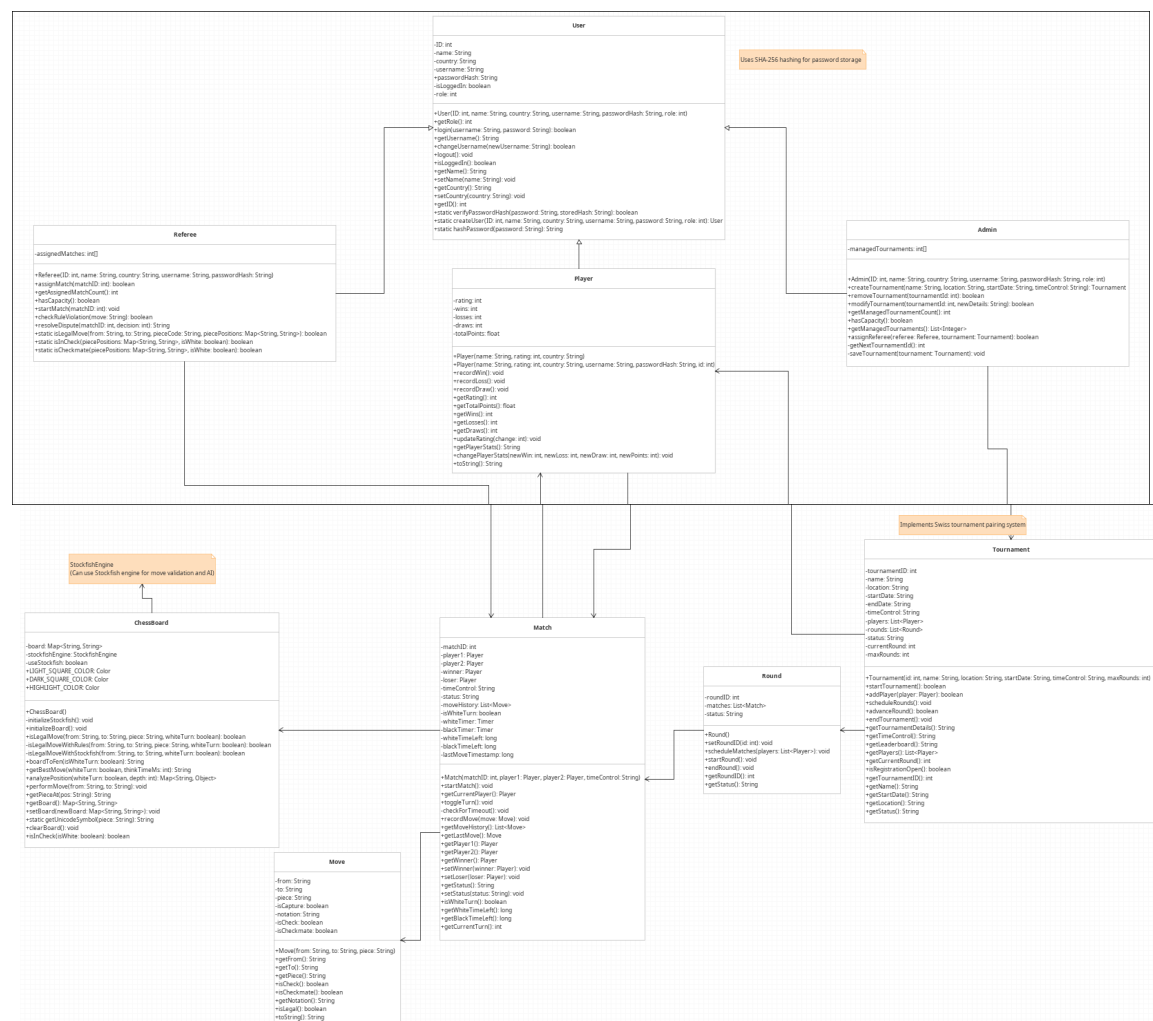
The Chess Tournament Management System addresses these challenges through a carefully designed architecture that divides responsibilities among different modules and provides appropriate tools for each user type. By digitalizing and automating the most complex aspects of tournament

management, the system reduces the administrative burden, minimizes errors, and enhances the experience for all participants.

III. Core Design

The project follows an object-oriented design and is built in Java. The main functionality revolves around managing and simulating chess tournaments, including user management (Admin, Player, Referee), tournament lifecycle management, and GUI components for interaction.

Core Model Classes & Tournament Logic



Main Classes:

User.java: Base class for all users. It includes shared properties like ID, name, country, username, and passwordHash.

Admin.java: Extends **User**. Admins can create, remove, and modify tournaments. They can also assign referees and track tournaments they manage.

Player.java: Represents a chess player with specific capabilities tied to playing matches and recording results.

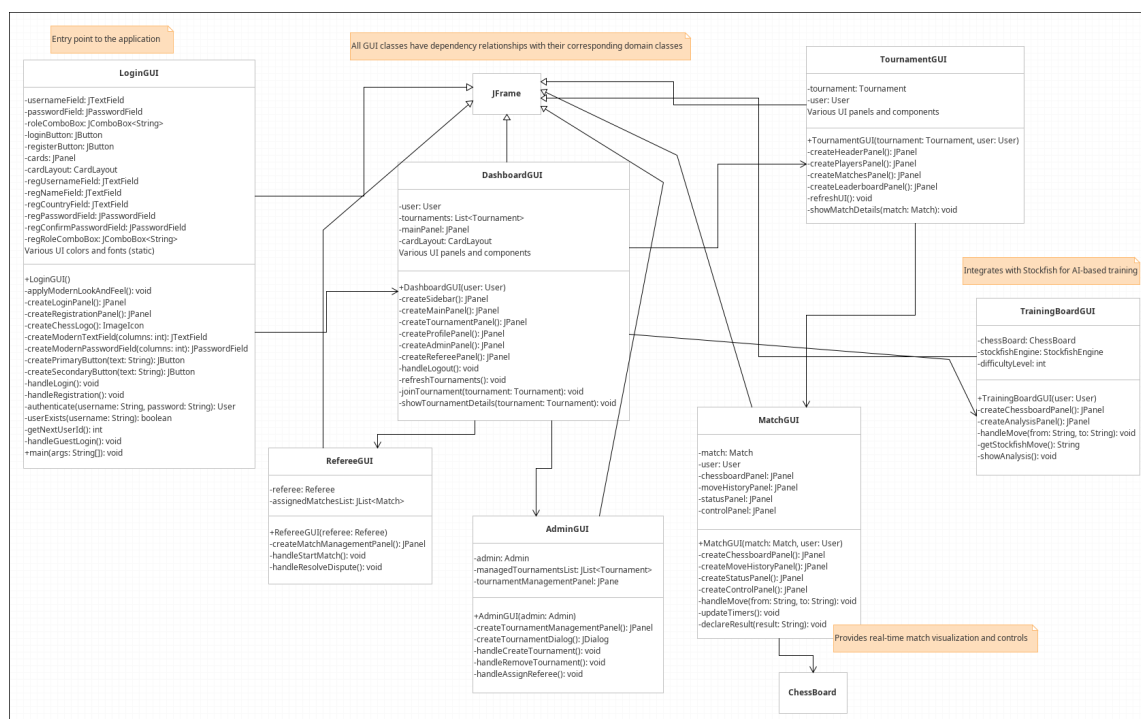
Referee.java: Referees oversee matches and are assignable by admins. They manage match outcomes and ensure fairness.

Tournament.java: Handles tournament-specific data, such as name, location, date, and time control. Admins can create these.

Match.java / Round.java: Represents individual matches and rounds within a tournament. These classes coordinate players and manage results.

ChessBoard.java / Move.java: Handle game logic, including player moves and board state. These are essential for simulating games.

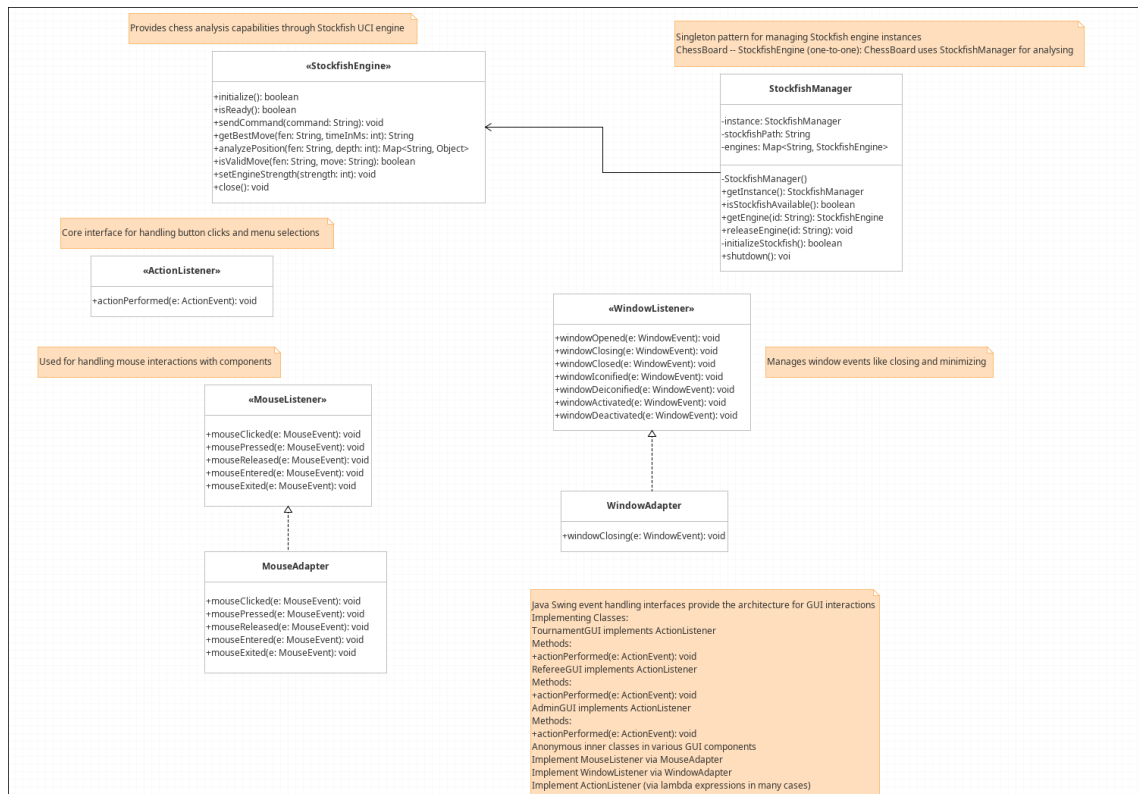
GUI Components:



GUI classes

LoginGUI.java / DashboardGUI.java / MatchGUI.java / TournamentGUI.java / TrainingBoardGUI.java / AdminGUI.java / RefereeGUI.java:

These provide a graphical interface for different roles to interact with the system. Each role has its own GUI for role-specific functionality.



Engine interface

StockfishEngine.java / StockfishManager.java: Integrates with Stockfish, a powerful chess engine. These classes are used to evaluate positions and simulate.

IV. Case Study

Scenario: An Admin manages a chess tournament.

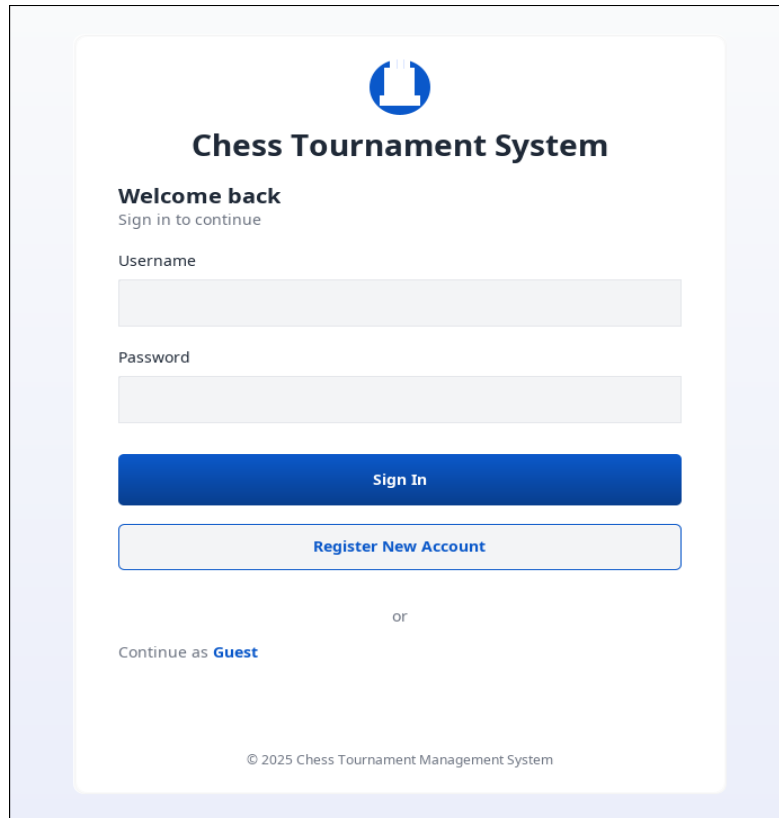
Admin Login: Admin logs in through LoginGUI.java, which authenticates using their username and passwordHash.

```
try {
    User user = authenticate(username, password);
    if (user != null) {
        JOptionPane.showMessageDialog(this
```

this is how credentials are checked:

```
private User authenticate(String username, String password) throws IOException {
    File usersFile = new File("users.txt");
    if (!usersFile.exists()) {
        return null;
    }
}
```

GUI:





Chess Tournament System

Welcome back
Sign in to continue

Username

Password

Sign In

[Register New Account](#)

or

Continue as [Guest](#)

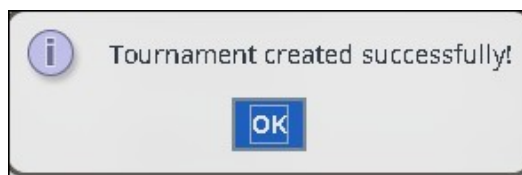
© 2025 Chess Tournament Management System

Tournament Creation:

The Admin creates a new tournament via AdminGUI.java using this method in Admin.java

```
// Create a new tournament
public Tournament createTournament(String name, String location, String startDate, String timeControl) {
    int nextTournamentId = getNextTournamentId();
    Tournament tournament = new Tournament(nextTournamentId, name, location, startDate, timeControl, 5);
}
```

Chess Tournament System					Administrator (Administrator)
- Chess TMS - Dashboard Tournaments - Profile - Admin Panel - Training Board	Available Tournaments				Create Tournament
	Name	Location	Date	Status	Players
	City Chess Championship	London	2023-11-05	upcoming	1
	Shanghai Masters	上海 (Shanghai)	2023-12-15	open	8
	Beijing Open	北京 (Beijing)	2024-01-20	open	12
	Guangzhou Invitational	广州 (Guangzhou)	2024-02-10	upcoming	6
	Hangzhou Cup	杭州 (Hangzhou)	2024-03-05	upcoming	4
	Chengdu Chess Festival	成都 (Chengdu)	2024-04-12	upcoming	0
	Nanjing Tournament	南京 (Nanjing)	2024-05-18	upcoming	2
	Shenzhen Championship	深圳 (Shenzhen)	2024-06-25	upcoming	3
	Xi'an Ancient City Cup	西安 (Xi'an)	2024-07-30	upcoming	0
	Suzhou Masters	苏州 (Suzhou)	2024-08-15	upcoming	5
View Details Join Tournament					Select a tournament to view details or join



(note: only the Admin is allowed to add tournaments)

after calling Admin.createTournament(...), which:

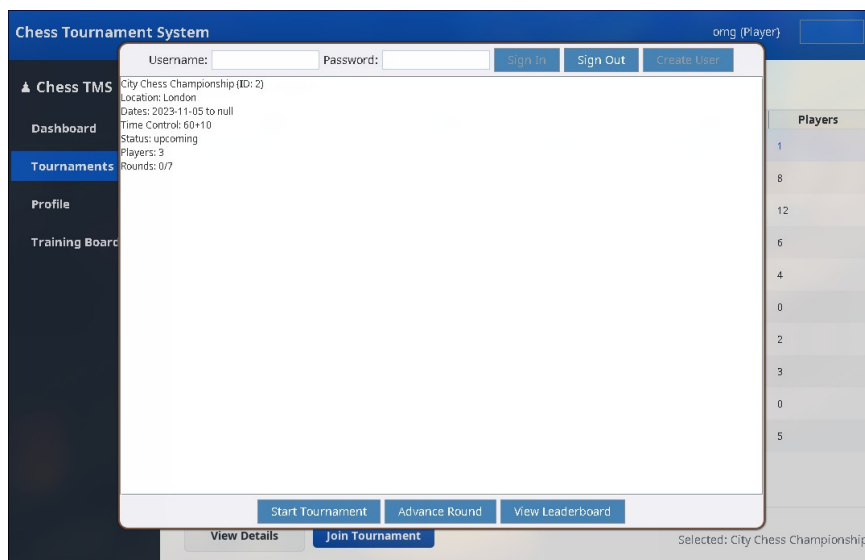
Generates a new ID with getNextTournamentId()

Initializes a new Tournament object.

Saves it (placeholder saveTournament() method) and assigns the tournament ID to the admin's managed list.

```
saveTournament(tournament);
return tournament;
}
```

After saving tournament you should be able to see details about it:



Assigning Referee:

Admin selects a referee.

Uses Admin.assignReferee(...) to associate the referee with the tournament: (This logs the action for traceability)

```
// Assign a referee to a tournament
public boolean assignReferee(Referee referee, Tournament tournament) {
    // In a real system, would update tournament-referee association in database
    System.out.println("Referee " + referee.getUsername() + " assigned to tournament "
        + tournament.getTournamentID() + " by admin " + getUsername());
    return true;
}
```

Running the Tournament:

```
public void scheduleRounds() {
    // Swiss tournament pairing
    Collections.sort(players, Comparator.comparingInt(Player::getRating).reversed());

    for (int i = 0; i < maxRounds; i++) {
        Round round = new Round();
        round.setRoundID(i + 1);

        List<Player> pairedPlayers = new ArrayList<>();
        List<Player> availablePlayers = new ArrayList<>(players);

        while (availablePlayers.size() >= 2) {
            Player player1 = availablePlayers.remove(0);
            Player player2 = findBestOpponent(player1, availablePlayers, pairedPlayers);

            if (player2 == null) {
                // No suitable opponent found, give bye to player1
                System.out.println("No suitable opponent found for " + player1.getName() + ", giving bye.");
                break; // or handle bye situation differently
            }

            availablePlayers.remove(player2);
            pairedPlayers.add(player1);
            pairedPlayers.add(player2);

            // In real system would create matches between player1 and player2
            round.scheduleMatches(players);
        }

        rounds.add(round);
    }
}
```

Matches and rounds are handled by Match.java and Round.java

```
public void scheduleMatches(List<Player> players) {
    matches.clear();
    long currentTime = System.currentTimeMillis();

    int matchId = 1000;

    for (int i = 0; i < players.size(); i += 2) {
        if (i + 1 < players.size()) {
            Player player1 = players.get(i);
            Player player2 = players.get(i + 1);
            Match match = new Match(matchId++, player1, player2, "10|0");
            matches.add(match);
            System.out.println("Scheduled match: " + player1.getUsername() + " vs " + player2.getUsername());
        } else {
            System.out.println("No opponent for: " + players.get(i).getUsername() + " - given a bye.");
        }
    }

    System.out.println("Scheduled " + matches.size() + " matches for round " + roundID);
}
```

this method creates match pairings for a round

Moves are processed using Move.java:

Move(from: String, to: String, piece: String) is Constructor

toString() is For displaying move textually:

```
public Move(String from, String to, String piece, String timestamp, boolean legal, Player player) {
    this.from = from;
    this.to = to;
    this.piece = piece;
    this.timestamp = timestamp;
    this.legal = legal;
    this.player = player;
}
```

and visualized through the ChessBoard.java: (we validate move using Stockfish)

```
public boolean isLegalMove(String from, String to, String piece, boolean whiteTurn) {
    if (useStockfish) {
        return isLegalMoveWithStockfish(from, to, whiteTurn);
    } else {
        return isLegalMoveWithRules(from, to, piece, whiteTurn);
    }
}
```



Integration with Engine:

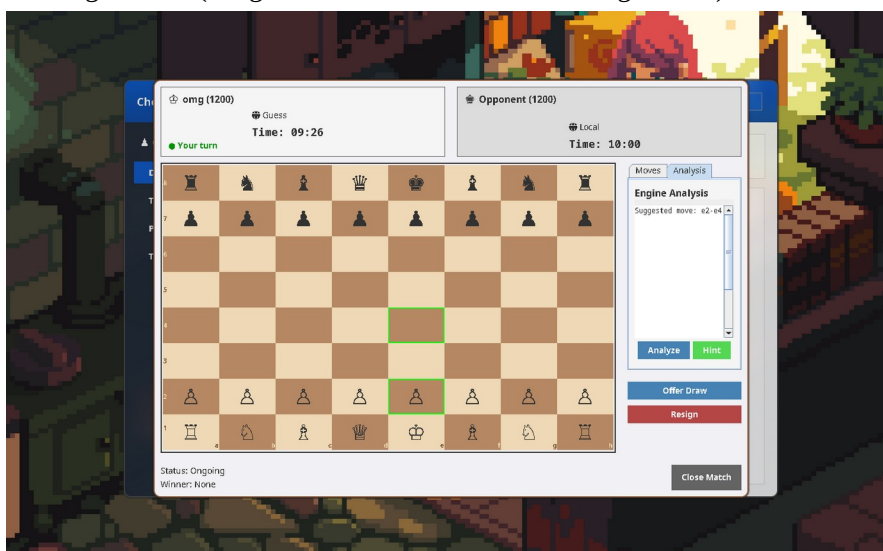
The StockfishManager or StockfishEngine is optionally used to analyze games or help during training mode via TrainingBoardGUI. (ChessBoard.java)

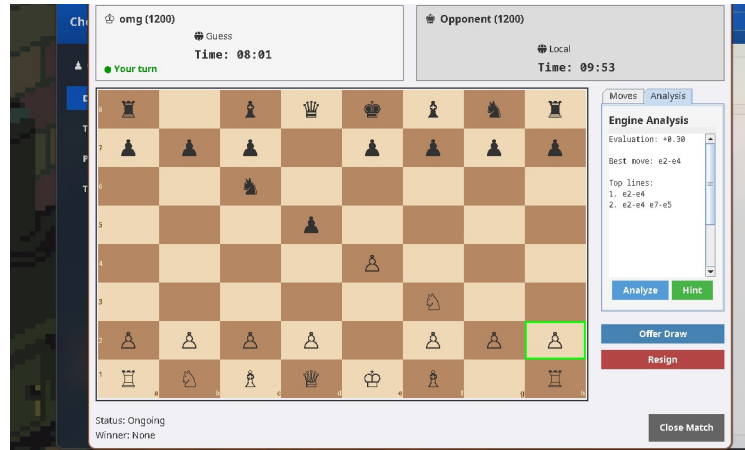
```
public String getBestMove(boolean whiteTurn, int thinkTimeMs) {
    if (!useStockfish) return null;

    String fen = boardToFen(whiteTurn);
    return stockfishEngine.getBestMove(fen, thinkTimeMs);
}
```

This queries stockfish to suggest a move

This illustrates the interaction among classes and how the system supports a full tournament lifecycle with admin oversight. (image below showcases stockfish engine hint)





Game Analysis Showcase (stockfish)

V. Contribution of Group Members

This project was developed collaboratively by three group members: Hana, Yasmine, and Raphael. We shared responsibilities throughout the development process, with Hana and Yasmine working interchangeably on both the core logic classes and the graphical user interface. While Raphael focused on the engine integration, implementing the interface between our application and the Stockfish chess engine to enable game analysis and training functionality. Together, we were responsible for building and integrating user facing components with the underlying tournament and gameplay systems, including features like user login, match scheduling, and move handling. Throughout the project, we maintained close communication, reviewed each other's work, and ensured that all components functioned smoothly together to deliver a cohesive and fully functional chess tournament management system.

VI. Conclusion:

The chess tournament management system is a comprehensive Java-based project built using object oriented principles. It supports multiple user roles (Admin, Player, Referee), each with tailored functionality and user interfaces. Admins can create, manage, and remove tournaments, assign referees, and interact with game data. The inclusion of Stockfish integration shows forward thinking for AI and chess analysis.

We tried to make this project as scalable and modular as possible, with a strong base for future enhancements like persistent storage, player ranking systems, or online multiplayer support. Overall, it demonstrates solid understanding of Java programming, GUI design, and software architecture.